

Procesamiento Distribuido de Video Mediante MPI en un Clúster Linux

Por: Juan David Prieto Velasquez

1. Introducción

El procesamiento de grandes cantidades de información puede beneficiarse del uso de sistemas distribuidos o compartidos. Y en este proyecto se implementó una solución basada en un clúster de servidores Linux para realizar tareas de procesamiento de video de forma paralela utilizando MPI (Message Passing Interface) y otras herramientas más.

La solución que se desarrolló permite dividir ocho videos originales (con resoluciones y fotogramas similares) en segmentos de diez segundos y, posteriormente, generar collages de los videos utilizando dichos segmentos. Así que para lograrlo se empleó una arquitectura maestro-trabajador (o master-worker en inglés), donde un nodo principal reparte las tareas entre varios nodos de procesamiento (también conocidos como workers).

El desarrollo fue realizado usando Python, y se utilizaron herramientas especializadas para la comunicación entre procesos y el procesamiento multimedia.

2. Objetivos

Objetivo General

Implementar un sistema distribuido capaz de segmentar videos y generar collages mediante procesamiento paralelo utilizando MPI.

Objetivos Específicos

- Configurar un clúster de servidores Linux para la ejecución distribuida de tareas.
- Implementar comunicación entre nodos utilizando MPI.
- Dividir videos de entrada en segmentos de diez segundos.
- Generar collages de video a partir de los segmentos obtenidos.

- Almacenar los resultados en un sistema de archivos compartido.
- Verificar el correcto funcionamiento del sistema distribuido.

3. Arquitectura de la Solución

La solución implementada utiliza una arquitectura master-worker compuesta por cinco máquinas virtuales.

El nodo principal (master) se encarga de coordinar la ejecución del sistema, es decir, distribuye las tareas y supervisa el procesamiento. Los cuatro nodos restantes funcionan como workers encargados de ejecutar las tareas asignadas.

Todos los nodos comparten un directorio común mediante NFS, lo que permite acceder a los mismos archivos desde cualquier máquina del clúster.

La distribución y los hostnames de estos nodos son las siguiente:

- Nodo Maestro: prieto-main
- Worker 1: prieto1
- Worker 2: prieto2
- Worker 3: prieto3
- Worker 4: prieto4

4. Herramientas Utilizadas

Las principales herramientas empleadas fueron:

- Ubuntu Server como sistema operativo.
- Python 3 para el desarrollo de los scripts.
- MPICH como implementación de MPI.
- mpi4py para utilizar MPI desde Python.
- FFmpeg para el procesamiento de video.
- NFS para compartir archivos entre nodos.
- Microsoft Azure para alojar las máquinas virtuales.

5. Configuración del Entorno

Inicialmente se me entregaron cinco máquinas virtuales dentro de Azure. Posteriormente se estableció conectividad mediante SSH entre todos los nodos para permitir la ejecución remota de procesos sin contraseña (o passwordless).

Se configuró además un sistema de almacenamiento compartido utilizando NFS. Este sistema permitió que todos los workers pudieran acceder simultáneamente a

las carpetas o directorios donde se encuentran archivos como los videos originales, segmentos generados, scripts de Python, y collages finales.

Durante la implementación se detectaron limitaciones de memoria en las máquinas virtuales (normalmente estas contaban alrededor de 300 a 400mb de memoria RAM), por lo que fue necesario habilitar memoria de intercambio (swap, y específicamente de 1GB) para garantizar el correcto funcionamiento de MPICH y FFmpeg. Además de poder instalar dependencias y las herramientas correctamente evitando problemas por falta de memoria.

6. Segmentación Paralela de Videos

La primera etapa consistió en dividir ocho videos (en este caso, videos gratuitos de stocks, los cuales se consiguieron en páginas como pixabay) en segmentos de diez segundos.

Para simplificar la generación de collages posteriores, todos los videos fueron normalizados para utilizar la misma resolución y la misma tasa de fotogramas, que en mi caso, elegí que tengan resoluciones de 640 x 360 y 30 fotogramas por segundo (técnicamente tienen 29.97 fotogramas cada uno).

Cada video fue procesado para obtener doce segmentos de diez segundos, y debido a que varios videos superaban los 2 minutos, se recurre a utilizar únicamente los primeros 120 segundos de duración.

La distribución de trabajo fue realizada por el nodo maestro de la siguiente forma:

- Worker 1: video1 y video5
- Worker 2: video2 y video6
- Worker 3: video3 y video7
- Worker 4: video4 y video8

Cada worker ejecutó FFmpeg sobre los videos asignados y generó sus respectivos segmentos.

Como resultado se obtuvieron:

8 videos × 12 segmentos = 96 segmentos de video.

Este proceso por lo general dura entre 2 a 3 minutos.

7. Generación Paralela de Collages

Una vez generados los segmentos, se procedió a crear los collages.

La construcción de los collages siguió la regla establecida en el enunciado:

- El collage 1 utiliza el segmento 1 de cada video.
- El collage 2 utiliza el segmento 2 de cada video.
- El proceso continúa hasta el collage 12.

Cada collage contiene simultáneamente los ocho videos organizados en una cuadrícula de cuatro columnas por dos filas (4x2).

La generación de los collages también fue distribuida entre los workers, asignando tres collages a cada nodo de procesamiento.

Esta estrategia permitió aprovechar los recursos disponibles y reducir el tiempo total de ejecución, que en este caso es menor a la generación de segmentos. Pues la duración de esta generación parte desde los 50 segundos a 1:30 minutos

8. Resultados

El sistema logró completar exitosamente todas las tareas requeridas.

Se generaron:

- 96 segmentos de video.
- 12 collages finales.
- Distribución automática de trabajo entre cuatro workers.
- Almacenamiento centralizado de resultados mediante NFS.

Durante las pruebas realizadas se verificó que los segmentos se generaban correctamente y que los collages mostraban simultáneamente los ocho videos correspondientes a su punto de partida (por ejemplo, el collage 1 mostraba los 8 videos desde 0 a 10 segundos, luego el collage 2 desde 10 a 20, collage 3 desde 20 a 30, y así). Además cabe destacar que los resultados de cada collage no son 10 segundos exactos, esto debido a los 29,97 fotogramas por segundo que manejaba cada video.

La ejecución distribuida permitió procesar múltiples videos de manera concurrente, demostrando el funcionamiento correcto de la arquitectura implementada.

9. Dificultades Encontradas

Durante el desarrollo del proyecto se presentaron diversas dificultades técnicas.

Inicialmente se encontraron problemas con la instalación de MPICH debido a dependencias incompletas dentro de algunas máquinas virtuales. La razón principal de esto (y otro de los problemas) es por la memoria RAM, puesto que las máquinas

tenían disponible una cantidad limitada (entre 300 a 400 MB mencionados previamente), lo que ocasionaba interrupciones durante la instalación de ciertos paquetes.

Para solucionar este problema se configuró memoria swap en cada nodo (y como se mencionó antes, se eligió 1GB para cada máquina)

También fue necesario verificar la comunicación SSH entre las máquinas y asegurar que todos los workers tuvieran acceso al sistema de archivos compartido.

Y finalmente, durante la generación de los collages fue necesario ajustar la distribución visual de los videos para garantizar que los ocho segmentos fueran visibles simultáneamente dentro del resultado final. Debido a que el resultado inicial presentaba un grid 2x2 mostrando 4 vídeos adelante, y 4 videos detrás de otros (es decir, los videos se superponen y por ende no se ven el collage completo).

10. Conclusiones

La implementación desarrollada permitió aplicar conceptos de computación distribuida mediante MPI en un escenario práctico como el procesamiento de videos.

La utilización de múltiples workers permitió dividir la carga de trabajo y ejecutar tareas en paralelo, reduciendo el tiempo requerido para procesar los videos.

El uso combinado de Python, MPI y FFmpeg demostró ser una solución flexible y eficiente para resolver problemas de procesamiento distribuido, teniendo en cuenta la limitada capacidad de las máquinas.

Finalmente, el proyecto permitió comprender de manera práctica la administración de clústeres, la comunicación entre nodos y la coordinación de tareas en sistemas distribuidos.

Referencias IEEE

[1] MPICH Team, “MPICH: A High Performance Portable MPI Implementation.” [Online]. Available: <https://www.mpich.org/>. [Accessed: 03-Jun-2026].

[2] MPI Forum, “MPI: A Message-Passing Interface Standard.” [Online]. Available: <https://www.mpi-forum.org/>. [Accessed: 03-Jun-2026].

[3] FFmpeg Developers, “FFmpeg Documentation.” [Online]. Available: <https://ffmpeg.org/documentation.html>. [Accessed: 03-Jun-2026].

[4] Python Software Foundation, “Python Documentation.” [Online]. Available: <https://docs.python.org/3/>. [Accessed: 03-Jun-2026].

Proyecto final de Estructura del Computador I

[5] L. Dalcin, “mpi4py Documentation.” [Online]. Available: <https://mpi4py.readthedocs.io/>. [Accessed: 03-Jun-2026].

[6] Canonical Ltd., “Ubuntu Server Documentation.” [Online]. Available: <https://ubuntu.com/server/docs>. [Accessed: 03-Jun-2026].

[7] Red Hat, Inc., “Network File System (NFS) Documentation.” [Online]. Available: <https://docs.redhat.com/>. [Accessed: 03-Jun-2026].

[8] Microsoft Corporation, “Azure Virtual Machines Documentation.” [Online]. Available: <https://learn.microsoft.com/azure/virtual-machines/>. [Accessed: 03-Jun-2026].